Configure Cryptographic Protocols

TLS · HTTPS · Secure Channels · SSH Key-Based Authentication · Wireshark Analysis

How the HVMC identity platform protects data in transit: configured, tested and analysed on the real Microsoft Azure / Entra ID environment, with an SSH implementation plan for the on-premises Samba server.

Project	AccessGuard Health — Horizon Valley Medical Center (HVMC) Healthcare IAM
Prepared By	Sabitha S Karanavar
Date	November 5, 2026
Environment	Microsoft Entra ID tenant (HVMC) and Azure subscription; capture taken on the lab Linux VM (192.168.64.30, Firefox) during a help-desk sign-in
Evidence	Wireshark capture `microsoft-login-dump.pcap` (79 TLS connections decoded), live openssl/curl tests, Azure resource settings
Cost	$0 (read-only tests; no new resources)

1. Approach and rubric coverage

Instead of building a separate web server, this report uses the identity platform HVMC already runs in the cloud. Every sign-in to Microsoft Entra ID, every Microsoft Graph call made by the IAM scripts, and every request to the HVMC storage account and Key Vault travels over TLS. The goal is to show **where** each protocol is implemented, **which settings HVMC controls**, and **what an observer on the network can and cannot see**.

Rubric requirement	Where it is shown	Status
Configuration of TLS for a web server	Section 2: HVMC storage account set to minimum TLS 1.2 + HTTPS only, tested live; Key Vault and Entra endpoints tested	Done
Implementation of a secure communication channel	Section 3: the help-desk sign-in to Entra ID end to end (DNS → TCP → TLS 1.3 → HTTP/2), what the channel protects	Done
Analysis of protocol security using Wireshark	Section 4: 79 TLS handshakes decoded from the capture — versions, cipher suites, key exchange, certificates, what stays visible	Done
Setup of SSH with key-based authentication	Section 5: implementation plan for the Ubuntu Samba AD server hvmc-dc01 (script prepared in the home lab kit)	Planned

2. TLS configuration on HVMC's web endpoints

In Azure the web server is a managed service, so TLS is configured as a **setting on the resource** rather than in an nginx or Apache file. HVMC's storage account `hvmcehrstorage` (simulated EHR documents) was configured during the cloud security work with these settings, read back from Azure on 5 November:

Setting (hvmcehrstorage)	Value	What it does
Minimum TLS version	**TLS 1.2**	Requests over TLS 1.0 or 1.1 are rejected
Secure transfer required (HTTPS only)	**Enabled**	Plain HTTP requests are rejected
Anonymous (public) blob access	**Disabled**	Every request must be authenticated, even over TLS
Certificate	*.blob.core.windows.net, issued by Microsoft TLS G2 RSA CA	Managed and renewed by Microsoft

The same settings are on the log storage account `hvmcseclogs…`. Key Vault (`hvmc-accessguard-kv`) and the Entra ID sign-in service enforce TLS 1.2+ by design.

Live tests (5 November 2026, from the office Mac)

```
TLS tests run from the office Mac on 5 Nov 2026 (read-only, $0)
=== handshake test: hvmcehrstorage.blob.core.windows.net
  tls1_3: ACCEPTED (TLSv1.3, AEAD-AES256-GCM-SHA384)
  tls1_2: ACCEPTED (TLSv1.2, ECDHE-RSA-AES256-GCM-SHA384)
  tls1_1: ACCEPTED (TLSv1.1, ECDHE-RSA-AES256-SHA)
  tls1: ACCEPTED (TLSv1, ECDHE-RSA-AES256-SHA)
=== handshake test: hvmc-accessguard-kv.vault.azure.net
  tls1_3: ACCEPTED (TLSv1.3, AEAD-AES256-GCM-SHA384)
  tls1_2: ACCEPTED (TLSv1.2, ECDHE-RSA-AES256-GCM-SHA384)
  tls1_1: REFUSED (no cipher agreed)
  tls1: REFUSED (no cipher agreed)
=== handshake test: login.microsoftonline.com
  tls1_3: ACCEPTED (TLSv1.3, AEAD-AES256-GCM-SHA384)
  tls1_2: ACCEPTED (TLSv1.2, ECDHE-RSA-AES256-GCM-SHA384)
  tls1_1: REFUSED (no cipher agreed)
  tls1: REFUSED (no cipher agreed)
=== request test: storage account with minimum TLS 1.2 (enforced per request)
  TLS 1.0 -> HTTP 400 The TLS version of the connection is not permitted on this storage account.
  TLS 1.1 -> HTTP 400 The TLS version of the connection is not permitted on this storage account.
  TLS 1.2 -> HTTP 409 Public access is not permitted on this storage account.
=== plain HTTP to the storage account (HTTPS only), run 5 Nov 2026 16:4x
  http:// -> HTTP 400 The account being accessed does not support http.
=== plain HTTP to login.microsoftonline.com
  HTTP/1.1 302 Found
  Location: https://login.microsoftonline.com:443/
client: LibreSSL 3.3.6, curl 8.7.1
```

Figure 1. TLS tests with openssl s_client and curl. The storage account still completes an old TLS 1.0/1.1 handshake at Azure's shared front end, but HVMC's minimum-TLS setting rejects the request itself with HTTP 400; TLS 1.2 passes the TLS check and is stopped only because the request had no credentials (409, public access off). Plain HTTP is refused. Key Vault and the sign-in service refuse TLS 1.0/1.1 at the handshake.

> **Finding T1 — where the minimum TLS version is enforced.** For Azure Storage, “minimum TLS 1.2” is enforced *per request*, not at the handshake: a scanner that only checks the handshake would wrongly report TLS 1.0 as enabled. Testing with a real request proved the setting works. This is why TLS settings must be tested end to end.

3. The secure communication channel: a help-desk sign-in

The capture was taken while the help-desk user signed in to `mysignins.microsoft.com` on 5 November (97 seconds, 4,354 packets). Every step that carried credentials, MFA approval and tokens went through an encrypted channel:

Step	What happens on the network	Protected?
1. Name lookup	DNS query for login.microsoftonline.com (first at t = 13.2 s), plain UDP port 53	**No**
2. TCP connection	3-way handshake to Microsoft's server on port 443	—
3. TLS 1.3 handshake	ClientHello/ServerHello agree TLS 1.3, AES-256-GCM, ECDHE key exchange; the server proves its identity with its certificate (encrypted)	**Yes**
4. Sign-in	User name, password, MFA push result and session cookies sent as HTTP/2 inside TLS	**Yes**
5. Tokens and API calls	Access tokens to graph.microsoft.com and api.mysignins.microsoft.com, again inside TLS	**Yes**

Microsoft endpoint (SNI)	Connections	TLS version	Cipher suite negotiated
mysignins.microsoft.com	23	TLS 1.3	TLS_AES_256_GCM_SHA384
aadcdn.msauth.net	13	TLS 1.3	TLS_AES_256_GCM_SHA384
login.microsoftonline.com	7	TLS 1.3	TLS_AES_256_GCM_SHA384
graph.microsoft.com	6	TLS 1.3	TLS_AES_256_GCM_SHA384
browser.events.data.microsoft.com	5	TLS 1.3	TLS_AES_256_GCM_SHA384
api.mysignins.microsoft.com	4	TLS 1.3	TLS_AES_256_GCM_SHA384
login.live.com	3	TLS 1.2	ECDHE-RSA-AES256-GCM-SHA384
res-1.cdn.office.net	2	TLS 1.3	TLS_AES_256_GCM_SHA384
webshell.suite.office.com	2	TLS 1.3	TLS_AES_256_GCM_SHA384
amcdn.msftauth.net	2	TLS 1.3	TLS_AES_256_GCM_SHA384
static2.sharepointonline.com	1	TLS 1.3	TLS_AES_256_GCM_SHA384
login.microsoft.com	1	TLS 1.3	TLS_AES_256_GCM_SHA384
api.myaccount.microsoft.com	1	TLS 1.3	TLS_AES_256_GCM_SHA384

Microsoft endpoint (SNI)	Connections	TLS version	Cipher suite negotiated
js.monitor.azure.com	1	TLS 1.3	TLS_AES_256_GCM_SHA384

Figure 2. Microsoft connections during the sign-in, decoded from the capture. The same channel properties protect the IAM PowerShell scripts, which call graph.microsoft.com.

What the channel guarantees: confidentiality (AES-GCM encryption), **integrity** (GCM authentication tag on every record), **server authentication** (certificate chained to a trusted root, checked by the browser) and **forward secrecy** (ephemeral ECDHE keys, so a stolen server key cannot decrypt this capture later). Plain HTTP is redirected to HTTPS and Entra sends HSTS for one year (3 November check), so the browser will not fall back to HTTP.

4. Wireshark analysis of the capture

79	73	6	0
TLS connections decoded	used TLS 1.3	used TLS 1.2	TLS 1.1 / 1.0 / SSL / plain HTTP

Negotiated protocol versions and cipher suites

TLS version	Connections	Share
TLS 1.3	73	92%
TLS 1.2	6	8%

Cipher suite	Connections	Share
TLS_AES_256_GCM_SHA384	69	87%
TLS_AES_128_GCM_SHA256	4	5%
ECDHE-RSA-AES128-GCM-SHA256	3	4%
ECDHE-RSA-AES256-GCM-SHA384	3	4%

Key exchange group	Connections	Share
secp256r1	65	82%
x25519	8	10%
(TLS 1.2: ECDHE in ServerKeyExchange)	6	8%

Figure 3. Every connection used an AEAD cipher (AES-GCM) with ephemeral elliptic-curve Diffie-Hellman. No CBC, RC4, 3DES or static-RSA suite was chosen.

Inside one handshake: login.microsoftonline.com

```
Frame 287  ClientHello  ->  login.microsoftonline.com  (20.190.157.9:443)
  supported_versions   : TLS 1.3, TLS 1.2
  cipher_suites (17)   : TLS_AES_128_GCM_SHA256, TLS_CHACHA20_POLY1305_SHA256, TLS_AES_256_GCM_SHA384
                         ECDHE-ECDSA-AES128-GCM-SHA256, ECDHE-RSA-AES128-GCM-SHA256, ECDHE-ECDSA-CHACHA20-
POLY1305, ECDHE-RSA-CHACHA20-POLY1305
                       ... plus legacy CBC / static-RSA suites: ECDHE-ECDSA-AES256-CBC-SHA, ECDHE-ECDSA-
AES128-CBC-SHA, ECDHE-RSA-AES128-CBC-SHA, ECDHE-RSA-AES256-CBC-SHA, RSA-AES128-GCM-SHA256, RSA-AES256-GCM-
SHA384, RSA-AES128-CBC-SHA, RSA-AES256-CBC-SHA
  supported_groups     : x25519, secp256r1, secp384r1, secp521r1, ffdhe2048, ffdhe3072
  key_share            : x25519, secp256r1   (ECDHE public keys sent up front)
  signature_algorithms : ecdsa_secp256r1_sha256, ecdsa_secp384r1_sha384, ecdsa_secp521r1_sha512,
rsa_pss_rsae_sha256, rsa_pss_rsae_sha384, rsa_pss_rsae_sha512, ...
  ALPN                 : h2, http/1.1
  server_name (SNI)    : login.microsoftonline.com   <- visible in clear text

Frame 306  ServerHello  <-  login.microsoftonline.com
  selected version     : TLS 1.3
  selected cipher      : TLS_AES_256_GCM_SHA384
  key_share            : secp256r1  (ephemeral ECDHE -> forward secrecy)
  after this point     : Certificate, CertificateVerify and Finished are ENCRYPTED (TLS 1.3)
Application data records seen (whole capture): client->server 473, server->client 638
```

Figure 4. Decoded ClientHello and ServerHello for the Entra ID sign-in service (Wireshark filter: tls.handshake.extensions_server_name == "login.microsoftonline.com").

Certificate chains (visible only on TLS 1.2 connections)

```
login.live.com  (TLS 1.2, ECDHE-RSA-AES256-GCM-SHA384) - certificate chain visible because TLS 1.2 sends it in
clear
  [1] CN=login.live.com, O=Microsoft Corporation          RSA 2048, sha256, valid 2026-10-20 -> 2027-01-28
      SAN: login.live.com, *.login.live.com
  [2] CN=DigiCert Global G2 TLS RSA SHA256 2020 CA1         RSA 2048, sha256, valid 2021-03-30 -> 2031-03-29
      issued by DigiCert Global Root G2 (trusted root in the browser/OS store)

firefox.settings.services.mozilla.com  (TLS 1.2, ECDHE-RSA-AES128-GCM-SHA256)
  [1] CN=firefox.settings.services.mozilla.com            RSA 2048, sha256, valid 2026-10-09 -> 2027-01-07
  [2] CN=YR1, O=Let's Encrypt  ->  [3] CN=Root YR, O=ISRG  ->  ISRG Root X1
```

Figure 5. In TLS 1.3 the certificate is encrypted, so it could only be read on the 6 TLS 1.2 connections (3 Certificate messages). Each chain ends at a root the client already trusts.

What an observer can and cannot see

Visible in the capture	Hidden by TLS
IP addresses and ports (who talked to which Microsoft server, and when)	User name, password and MFA approval
Server names: DNS queries and the TLS SNI field (19 distinct names)	Session cookies, access tokens and refresh tokens
TLS version, offered and chosen cipher suites, key-exchange group	Page content, Graph API requests and responses
Amount and timing of traffic	Even TLS alerts (4 seen) are encrypted; in TLS 1.3 the certificate is too

Findings

#	Finding	Risk	Recommendation
W1	73 of 79 connections used TLS 1.3; the rest TLS 1.2 with ECDHE + AES-GCM. No legacy protocol or cipher was negotiated.	Low	Keep minimum TLS 1.2 on every HVMC resource; prefer 1.3
W2	DNS queries and the SNI field show which services were used (e.g. login.microsoftonline.com) in clear text.	Low–Medium (privacy, metadata)	Encrypted DNS (DNS over HTTPS) on staff devices; Encrypted Client Hello where supported
W3	The browser still *offers* legacy CBC and static-RSA suites and SHA-1 signatures for compatibility; the servers never chose them.	Low (server-side policy prevents downgrade)	Managed browser policy for clinical workstations can remove legacy suites
W4	Some traffic used QUIC (HTTP/3 over UDP 443, 123 packets), which carries TLS 1.3 inside QUIC.	Info	Firewall rules and inspection must account for UDP 443, not only TCP 443
T1	Storage minimum TLS is enforced per request (section 2).	Info	Test with real requests, not only handshakes

5. SSH with key-based authentication on the Samba server (implementation plan)

Planned HVMC's on-premises identity server **hvmc-dc01** (Ubuntu with Samba as an Active Directory domain controller, 192.168.56.10) is administered over SSH. Cloud services do not use SSH for sign-in, so this part is implemented on the Samba server. The script `04_ssh_hardening.sh` in the home lab kit already automates the server side; this section describes what it does and how it will be verified.

Why keys instead of passwords on a domain controller

- A domain controller holds every user's credentials: a guessed or phished admin password would hand over the whole domain.
- With a key, the secret (private key) never leaves the admin's device and is never sent over the network; the server only receives a signature, so nothing reusable can be captured or brute-forced.
- Password login over SSH is a constant target of automated guessing; turning it off removes that attack entirely.

Steps

```
# 1. On the admin Mac: create the key pair (private key stays on the Mac, protected by a passphrase)
ssh-keygen -t ed25519 -a 100 -C "hvmc-iam-admin" -f ~/.ssh/hvmc_dc01_ed25519

# 2. Install the PUBLIC key on the server (uses the password one last time)
ssh-copy-id -i ~/.ssh/hvmc_dc01_ed25519.pub ubuntu@192.168.56.10

# 3. Prove the key works BEFORE turning passwords off
ssh -i ~/.ssh/hvmc_dc01_ed25519 ubuntu@192.168.56.10 'echo key login ok'

# 4. On the server: add the hardening file, validate, reload (existing sessions stay open)
sudo sshd -t && sudo systemctl reload ssh

# 5. Tests from the Mac
ssh -o PubkeyAuthentication=no ubuntu@192.168.56.10      # expect: Permission denied (publickey)
ssh root@192.168.56.10                                    # expect: Permission denied
ssh -i ~/.ssh/hvmc_dc01_ed25519 ubuntu@192.168.56.10     # expect: logged in, no password asked
```

Server configuration

```
# /etc/ssh/sshd_config.d/00-hvmc-hardening.conf  (hvmc-dc01, Ubuntu + Samba AD DC)
PubkeyAuthentication yes
AuthenticationMethods publickey
PasswordAuthentication no
KbdInteractiveAuthentication no
PermitEmptyPasswords no
PermitRootLogin no
AllowUsers ubuntu                # the local admin account only; domain users do not get a shell on the DC
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no
KexAlgorithms sntrup761x25519-sha512@openssh.com,curve25519-sha256,curve25519-sha256@libssh.org
Ciphers chacha20-poly1305@openssh.com,aes256-gcm@openssh.com,aes128-gcm@openssh.com
MACs hmac-sha2-512-etm@openssh.com,hmac-sha2-256-etm@openssh.com
```

Figure 6. Hardening file for hvmc-dc01. A separate file in sshd_config.d keeps the distribution's sshd_config untouched and is easy to undo.

Control	Why it matters on the Samba DC
Ed25519 key with passphrase (-a 100)	Modern, short keys; the passphrase protects the private key if the Mac is stolen
AuthenticationMethods publickey; PasswordAuthentication no	Only key logins are accepted; password guessing is impossible
PermitRootLogin no; AllowUsers ubuntu	Domain users (e.g. HVMC\sarah.chen) can sign in to Windows/Samba services but cannot open a shell on the DC
sntrup761x25519 / curve25519 key exchange; ChaCha20-Poly1305 / AES-GCM; ETM MACs	Same principles as the TLS analysis: forward secrecy, AEAD ciphers, a post-quantum hybrid key exchange
Host key fingerprint checked on first connect	Stops a man-in-the-middle impersonating the DC (the SSH equivalent of a TLS certificate check)
sshd -t before reload; keep the current session and the VirtualBox console open	A typo cannot lock the administrator out of the DC

How it interacts with Samba

SSH only controls **administrative shell access**. Samba's own services keep their own protection: Kerberos tickets and LDAP for domain sign-ins, and SMB 3 encryption for file shares (planned in the symmetric-

encryption add-on). Because `AllowUsers` names the local admin only, the hardening does not affect domain users or Samba services, and Samba's `winbind` login integration is not needed for SSH.

Verification and Wireshark

Test	Expected result
Key login	Logged in without a password prompt; journalctl shows `Accepted publickey for ubuntu`
Password-only login (-o PubkeyAuthentication=no)	`Permission denied (publickey)`
root login	Permission denied
`sudo sshd -T`	passwordauthentication no, permitrootlogin no, the algorithm lists above
Wireshark filter `ssh` while logging in	Only the two version banners (SSH-2.0-OpenSSH_…) and the KEXINIT algorithm lists are readable; after NEWKEYS every packet is encrypted, including the user name and the key signature

6. Cloud and on-premises side by side

Protocol need	Cloud (Entra ID / Azure)	On-premises (hvmc-dc01, Samba AD)
Web/API traffic	TLS 1.2/1.3 managed by Microsoft; HVMC sets minimum TLS and HTTPS-only per resource	LDAPS / TLS with a certificate from the HVMC CA (Rubric 4 PKI)
Administrator access	Microsoft Graph over TLS, MFA, least-privileged help-desk role	SSH with Ed25519 keys, no passwords, no root
Certificates	Public CAs (DigiCert, Microsoft TLS G2), renewed automatically	Private HVMC CA; HVMC renews and revokes
Who configures what	Microsoft runs the server side; HVMC chooses settings and verifies them	HVMC configures and maintains everything

7. Conclusion

All 79 TLS connections in the sign-in capture used TLS 1.2 or 1.3 with forward-secret key exchange and authenticated encryption, and nothing sensitive was readable. HVMC's own storage endpoint rejects TLS 1.0/1.1 and plain HTTP, proven with live requests. What remains visible is metadata (server names, timing), which encrypted DNS and ECH would reduce. SSH key-based access to the Samba domain controller is designed and scripted, and is the next item to run in the home lab.

Next steps: run `04_ssh_hardening.sh` on hvmc-dc01 and add the SSH capture to this report; optionally capture with an SSLKEYLOGFILE in a lab browser to show a decrypted view (the key file is a secret and is never published).

Appendix — reproducing the analysis in Wireshark

What to show	Wireshark display filter
All TLS handshakes	`tls.handshake.type == 1 || tls.handshake.type == 2`

What to show	Wireshark display filter
The Entra sign-in handshake	`tls.handshake.extensions_server_name == "login.microsoftonline.com"`
Negotiated TLS 1.3	`tls.handshake.extensions.supported_version == 0x0304 && tls.handshake.type == 2`
Certificates sent in clear (TLS 1.2)	`tls.handshake.type == 11`
DNS names looked up	`dns.flags.response == 0`
QUIC / HTTP/3	`quic`
Any plain HTTP (expected: none)	`http`

The capture was decoded with Python (Scapy) scripts that read the same fields Wireshark shows; they are kept with the evidence. The raw capture file is not published, because it contains device addresses and browsing metadata.